

Concurrence et partage de données

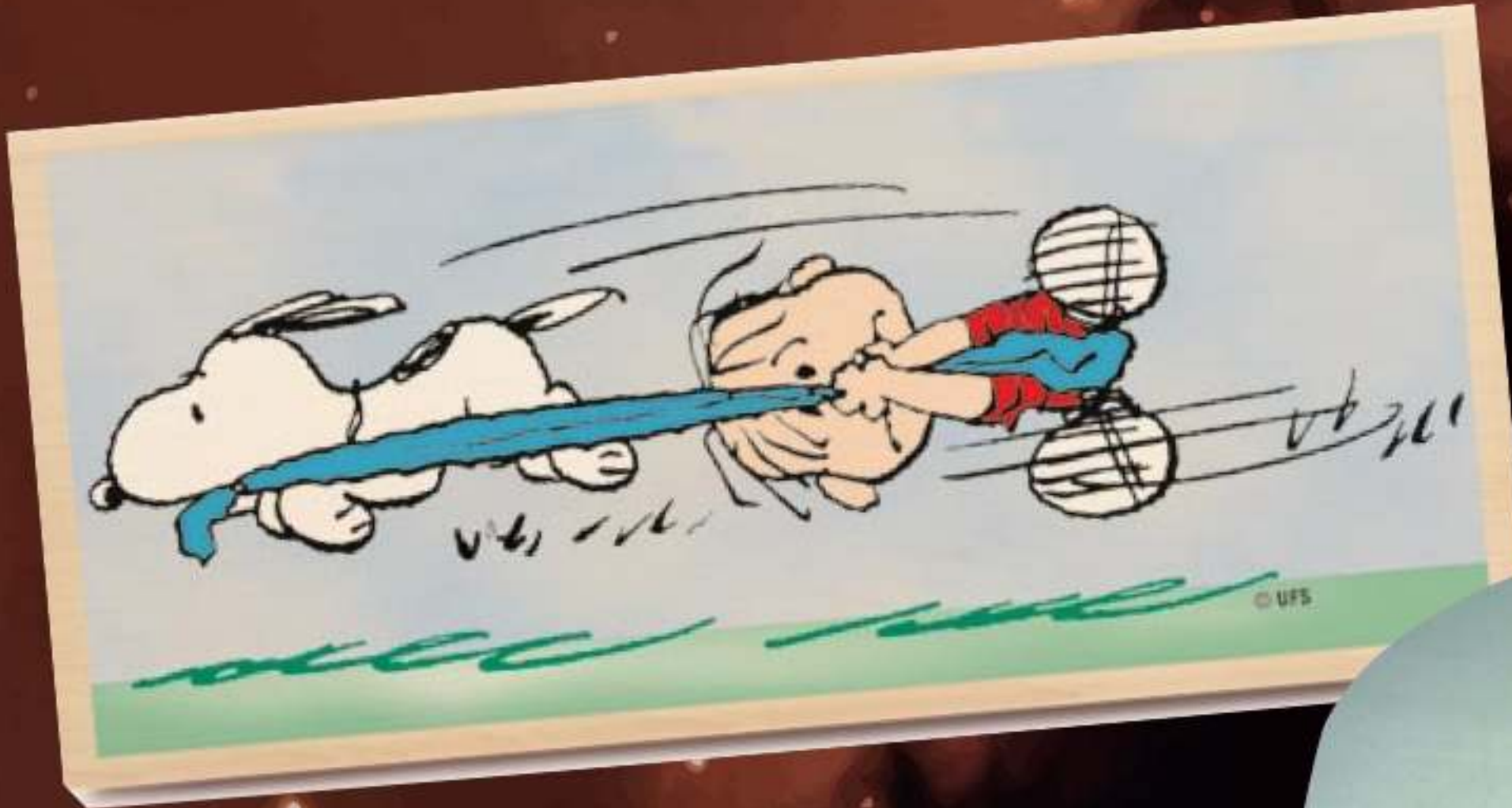
Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

Deux aspects du partage...



Par l'exemple du compteur partagé (classe Compteur)

3

```
class Compteur {  
    private String monNom;  
    private int compteur = 0;  
  
    Compteur (String n) {  
        monNom = n;  
    }  
  
    public void inc() {  
        compteur++;  
        System.out.println ("après inc, " + this.toString());  
    }  
  
    public void dec() {  
        compteur--;  
        System.out.println ("après dec, " + this.toString());  
    }  
  
    public String toString () {  
        return monNom + "=" + compteur;  
    }  
}
```


Le compteur partagé (classe Client)

4

```
class Client implements Runnable {
    private int monNumero;
    private Compteur monCpt;
    Client (int i, Compteur c) {
        monNumero = i;
        monCpt = c;
    }
    public void run() {
        System.out.println (monNumero + " est lancé");
        for (int i = 1 ; i <= 4 ; i++) {
            if (monNumero % 2 == 0) {
                monCpt.inc();
            } else {
                monCpt.dec();
            }
            try { // On met la dose pour forcer une commutation;-)
                Thread.sleep (100); // millisecondes
            }
            catch (InterruptedException ie) {
                System.err.println("Sleep impossible pour " + monNumero);
            }
        }
    }
    public String toString () {
        return "client " + monNumero;
    }
}
```


Le compteur partagé (classe «main»)

5

```
public class CompteurPartage1 {  
    public static void main(String []args) {  
        Compteur cpt = new Compteur("cpt");  
  
        Thread lesClients [] = new Thread [6];  
  
        for (int i = 0; i < lesClients.length ; i++) {  
            Client c = new Client (i+1, cpt);  
            lesClients [i] = new Thread(c);  
            lesClients[i].start();  
        }  
    }  
}
```


Le compteur partagé (classe «main»)

5

```
public class CompteurPartage1 {
```

```
$ java CompteurPartage1
```

```
1 est lancé  
6 est lancé  
après inc, cpt=0  
4 est lancé  
après inc, cpt=1  
5 est lancé  
après dec, cpt=0  
3 est lancé  
après dec, cpt=-1  
2 est lancé  
après inc, cpt=0  
après dec, cpt=-1  
après inc, cpt=1  
après inc, cpt=3  
après dec, cpt=2  
après dec, cpt=2  
après dec, cpt=2  
après inc, cpt=2  
après dec, cpt=2  
après inc, cpt=4  
après inc, cpt=4  
après dec, cpt=2  
après inc, cpt=2  
après dec, cpt=2  
après inc, cpt=4  
après inc, cpt=5  
après dec, cpt=5  
après dec, cpt=5  
après inc, cpt=5  
après inc, cpt=5
```

```
$ java CompteurPartage1
```

```
2 est lancé  
6 est lancé  
3 est lancé  
après dec, cpt=1  
5 est lancé  
après dec, cpt=0
```

```
$ java CompteurPartage1
```

```
1 est lancé  
5 est lancé  
après dec, cpt=-2  
3 est lancé  
après dec, cpt=-3  
4 est lancé  
après inc, cpt=-1  
après dec, cpt=-2  
après inc, cpt=0  
après dec, cpt=-1  
après inc, cpt=-1  
après dec, cpt=-1  
après inc, cpt=-1  
après dec, cpt=0  
après inc, cpt=0  
après dec, cpt=0  
après inc, cpt=-1  
après dec, cpt=0  
après inc, cpt=-1  
après dec, cpt=-1  
après inc, cpt=-1  
après dec, cpt=-1  
après inc, cpt=1  
après inc, cpt=1  
après dec, cpt=-1  
après dec, cpt=0  
après inc, cpt=0
```

Questions

Combien d'inc() sont-ils exécutés?

Combien de dec() sont-ils exécutés?

Ces nombres peuvent-ils varier?

3 exécutions, 3 résultats, est-ce normal?

```
après dec, cpt=-2  
après dec, cpt=-2  
après dec, cpt=-2  
après inc, cpt=-3  
après dec, cpt=-3  
après inc, cpt=-2  
après inc, cpt=-3  
après dec, cpt=-3  
après inc, cpt=-2
```

```
après inc, cpt=0
```

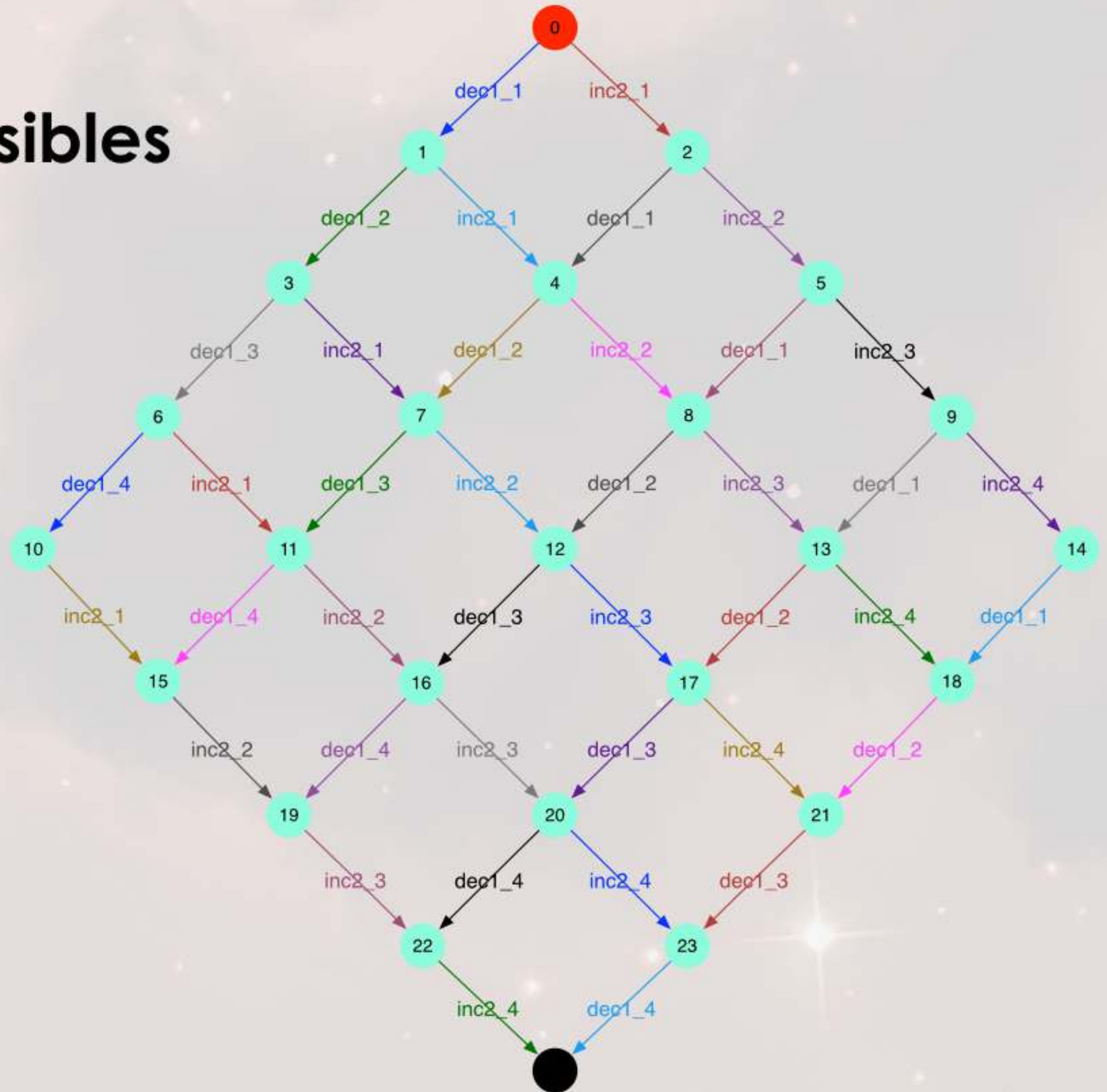

Analysons les exécutions possibles...

6

 Appréhender la notion d'entrelacement

 Les exécutions possibles

 Pour 2 clients (1 et 2)



Analysons les exécutions possibles...

6

Appréhen

Les exécutions

Pour 2 clients (1 et 2)

Où est le problème?

Les opérations ne sont pas indivisibles!

Cela explose vite!

2 clients	25 nœuds + 40 arcs
4 Clients	625 nœuds + 2 000 arcs
6 clients	15 625 nœuds + 75 000 arcs



La notion d'atomicité

7

```
public void inc() {
    compteur++;
}
```

```
public void dec() {
    compteur--;
}
```

Compilation/interprétation

```
MOV R, $adr
INC R
MOV $adr, R
```

```
MOV R, $adr
DEC R
MOV $adr, R
```



Exécutons...

thread 1				R	thread 2
	1	1	1	?	
MOV R, \$adr	1	1	1	?	
	1	1	1	1	MOV R, \$adr
	1	1	0	0	DEC R
	1	0	0	0	MOV \$adr, R
INC R	2	0	1	1	
MOV \$adr, R	2	2	1	1	
	⋮	⋮	⋮	⋮	

Commutation

Commutation

La notion d'atomicité

7

```
public void inc() {
    compteur++;
}
```

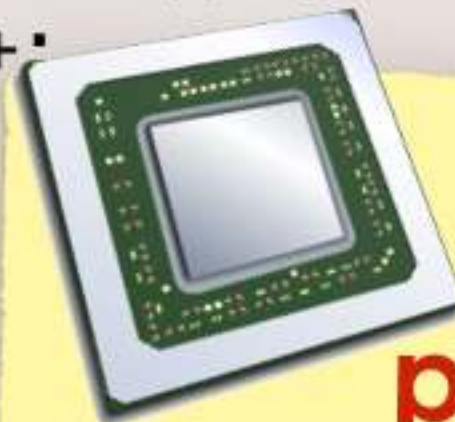
```
public void
compteur--
}
```

```
MOV R, $adr
INC R
```

r, R

\$adr

r, R



**Pire, dans les
processeurs modernes**

il existe du parallélisme à un niveau très fin
(«pipe-line», UAL dupliquées, etc.)



Exécutons...

thread 1

MOV R, \$adr

MOV \$adr, R

thread 2

Commutation

Le mal est fait!!!

Une opération est perdue

La bonne question

Quelle atomicité?

R

?

?

1

1

1

1

0

0

2

⋮

2

⋮

1

⋮

La notion de ressource

8

«chose» partagée

- Accès exclusif
- Opérations atomiques
 - ▶ Protection nécessaire



Accès aux ressources critiques?
Manipulation dans une **section critique**

Différents types de ressources

- Critique
 - ▶ Un exemplaire partagé par tous avec accès exclusif
- Banalisée
 - ▶ Différentes copies équivalentes possédées un temps par une entité

Construire des sections critiques?

- Synchronisation requise pour garantir l'atomicité des manipulations
 - ▶ Utiliser l'ordonnanceur (scheduler) de l'OS

Et si le réseau s'en mêle?

9

L'histoire du village Gaulois

Temps	Babaorum	Petibonum
1	Décision d'une attaque commune à 7	Vie tranquille au camp
2	Envoi d'un messenger vers Petibonum	Vie tranquille au camp
3	Préparatifs / messenger perdu	Vie tranquille au camp
4	Préparatifs / messenger perdu	Vie tranquille au camp
5	Sortie des troupes / messenger perdu	Vigie signalant activité à Babaorum
6	Troupes en route vers le village	Arrivée du messenger
7	Attaque repoussée par les gaulois	Préparatifs
8	Retraite (en désordre)	Préparatifs
9	Rentrée au camp	Sortie des troupes
10	Soigner plaies et bosses	«seconde tournée» pour les Gaulois
11	Soigner plaies et bosses	Retraite (en désordre)
12	Soigner plaies et bosses	Rentrée au camp

Généralisation, les «généraux byzantins»

Et si le réseau s'en mêle?

9

L'histoire du village G

Origine du problème

Communications
Durée et/ou erreurs

Temps	Babaorum	Pelbonum
1	Décision d'une	Vie tranquille au camp
2	Envoi d'un messenger vers Pelbonum	Vie tranquille au camp
3	Préparatifs / messenger perdu	Vie tranquille au camp
4	Préparatifs / messenger perdu	Vie tranquille au camp
5	Sortie des troupes / messenger perdu	Vigie signalant activité à Babaorum
6	Troupes en route vers le village	Arrivée du messenger
7	Attaque repoussée par les gaulois	Préparatifs
8	Retraite (en désordre)	Préparatifs
9	Rentrée au camp	Sortie des troupes
10	Soigner plaies et bosses	«seconde tournée» pour les Gaulois
11	Soigner plaies et bosses	Retraite (en désordre)
12	Soigner plaies et bosses	Rentrée au camp

Généralisation, les «généraux byzantins»

En guise de conclusion...

10

Problèmes

- L'exécution n'a pas l'atomicité que l'on souhaite
 - ▶ Test and set
 - ▶ Compare and swap
- Les communications troublent le jeu (sans hypothèses)
 - ▶ Pertes, délais

État global du système?

- C'est une notion qui existe...
- C'est une notion dénuée de sens
 - ▶ État global de l'humanité?

Le tout est compliqué par l'entrelacement

- Multiplicité des exécutions



En guise de conclusion...

10

Problèmes

• L'exécution n'a pas l'atomicité que l'on souhaite

► Test an

► Compo

• Les cor

► Pertes,

État g

• C'est u

• C'est une notion dénuée de sens

► État global de l'humanité?

Le tout est compliqué par l'entrelacement

• Multiplicité des exécutions

Conséquences?

Les systèmes concurrents sont **difficiles** et **instables**

La conception est difficile

problème des philosophes, 10^{4771} configurations
possibles pour 10000 convives

